claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_fc91ae6e-b62\agent-a5a02dfead8b71fab.jsonl`
- SHA-256: `e4f225658450cd2309de52a708c27bf1645074531981ec943c701d400ed78ba5`
- Source modified: `2026-06-14T06:51:36+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_fc91ae6e-b62`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T06:50:14.733Z)

In the repo (cwd), extract the PLANNED work + the corpus/labeling story. Read docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md, docs/NEXT_STEPS.md, docs/EXECUTION_POLICY.md, and docs/ROORA_LABELING_GUIDELINES.md (skim). Return: the owned-model roadmap (base = Qwen3-VL Apache-2.0 primary, InternVL3 MIT fallback; commercial-clean MIT/Apache/BSD-only licensing for code+data+weights; paid LLMs = inference/serving ONLY never training data); the execution-policy resilience framework (per-job wait/abort/resume, op_timeout, self-scheduling); and the concrete next experiments (grow the golden corpus; activate court polygons so the person/court-bounds cue stops counting spectators; measure audio on shuttle-only directly; a future "shuttle-dropped = shuttle-on-floor for a contiguous frame sequence" cue). ALSO frame the labeling vendor (Roora) as an ORACLE LABEL GENERATOR: the system's contract is with the golden labels (assumed trustable ground truth), NOT their provenance ? so evaluation rigor is decoupled from the vendor and the corpus scales by adding/swapping oracles. Capture the corpus = owner-grown, proprietary, minors excluded.

2. assistant (2026-06-14T06:50:16.990Z)

I'll read the four docs to extract the planned work and corpus/labeling story.

3. user (2026-06-14T06:50:17.925Z)

```
1       # Owned-model implementation plan (roadmap) ? DRAFT for owner review
2
3       **Status:** PLAN / roadmap. No owned-model *implementation* starts until the owner greenlights a specific
4       milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a training source).
5       **Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO + learned prototype
6       (`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy workflow (2026-06-09).
7
8       ## Design principle (owner directive): *envision now, build later*
9       Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be **architecturally envisioned
10      now** so we never hit a mammoth refactor. We **design the seams** (data contracts, export boundaries, the
11      event store) up front and **implement behind them** when each milestone is greenlit. Punting a *feature* is fine;
12      punting its *seam* is not.
13
14      ## North Star (reframed per PR #61 review)
15      - **Immediate deliverable:** a **training framework + harness** that produces a model with **very high precision/
16        recall for extracting highlights + rallies from a clip**. *Quality is the first-order
```

objective.*
17   - **Eventual product:** a **single, sport-agnostic, conversational** video model ? upload a clip, then ask
18   contextual questions ("longest rally", "make a clip of all service faults", "rallies over 20 shots").
19   - **On-device is LATER** (a model-compression goal so power users save upload cost/hassle) ? definitely on the
20   roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
21   - **The moat = the consented dataset + the eval harness, not the weights.**
22
23   ## Hard guardrails (non-negotiable ? "never corrupted")
24   - **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT / Apache-2.0 / BSD only.**
25   (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-restrictive: Gemma 3 (viral),
26   **Gemma 3n** (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU cap), Commons-Clause repos.
27   - **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public grounding-CoT datasets that
28   were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision (a real, binding cost).
29   - **Exclude minors; per-user training data needs a separate explicit opt-in; split data by match.**
30   - ? **Base-model pretraining provenance is unauditable for *every* open VLM (incl. Qwen3-VL).** Our "no paid-LLM
31   training" rule is enforceable at *our fine-tuning data* layer, not the base's pretraining ? an **explicit owner
32   risk-acceptance** is required to use any open VLM. (Governed by the base's license, not our pipeline.)
33
34   ## The model strategy (verified 2026-06-09)
35   **Framework ? base** (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge (runs SFT/GRPO, emits
36   weights); a *base* is the metal you forge. You pick one framework and feed it any clean base.
37   - **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio + timestamp-grounding +
38   GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core; *avoid its AGPL Studio UI*)
39   as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-time snapshot ? re-verify at impl.
40   - **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-video context, purpose-built
41   **timestamp emission** (second-level localization), multi-turn video chat. Re-confirm the exact size's LICENSE
42   pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but its Prohibited-Use posture is
43   unconfirmed (counsel needed), *and* its video/timestamp capability is disputed; not the primary extractor.
44   **InternVL3 (MIT/Apache)** = clean fallback if a Qwen blocker appears. ? base swap is cheap *only within
45   timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-specific).
46   - **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss strict temporal boundaries**
47   (best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse regions, not accurate boundaries").
48   **Boundary accuracy IS the product.** The head (`rally_seq_proto.py`) is already de-risked on our corpus (held-out
49   AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params (trivially compressible later),
50   and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So: **head now ? VLM later**,
51   strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes, head refines boundaries) over

```
52        wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur
footage may differ ? re-measure.)*
53
54      ## Conversational video-QA ? build the bookkeeping NOW, punt the model
55      **Punt the conversational *feature*** until extraction quality is solid (current LOVO
~0.583 ? "very high"; a chat UI
56      over a weak extractor exposes the weakness). **Reject end-to-end-VLM-answers-each-turn**
(expensive; bad at exact
57      counting; the strong teachers are license-forbidden). **But build the *seam* now** (the
brief requires it):
58      - **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits
structured timestamped
59        spans ? a durable **per-video EVENT STORE** ? a **structured QUERY layer** (text-to-
SQL + ffmpeg clip-cut). The
60        cited queries ("longest rally", "all service faults", "rallies > 20 shots") are
**structured-numeric ? SQL, not a
61        VLM job** (SQL counts exactly where VLMs hallucinate).
62      - **Designed-now seam:** make the **per-video event-index schema a first-class data
contract** alongside the M0.1
63        corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count,
ending_reason/fault_tags,
64        derived_clip_ref, provenance, confidence}`. **Reserve the event/fault-tag taxonomy
(e.g. `service_fault`) up front**
65        even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration):
`play_segments` today has
66        `video_id/start_time/end_time/duration/server/receiver/metadata` only; `highlights`
isn't a table yet ? so it's an
67        *extension with new columns + a highlights table*, not a no-op. **Punt** the NL
parser/agent until quality clears the bar.
68      - Paid Gemini may answer genuinely open-ended edge questions **at inference** over our
structured index (allowed ?
69        it reasons over our data, it is not trained on its outputs).
70
71      ## Multisport (single sport-agnostic model)
72      - **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned
across sports (sport = an input
73        feature / per-sport calibration). The collector + indexer are already sport-generic. ?
**ACTION: rename the repo off
74        "badminton" ASAP** (owner emphatic) ? consistent with this.
75      - **What multisport changes = DATA + DETECTORS (scale up, the accepted "good
problem"):** each new sport needs its own
76        labeled matches (split by match) + a **clean trajectory detector** to feed the head's
features. **The binding blocker
77        is per-sport detectors:** WASB (badminton) has the unresolved **C3** checkpoint-
provenance issue, and **clean MIT/
78        Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet
identified.** Sequence by public-data
79        richness: **badminton ? tennis/TT ? pickleball/padel.**
80      - ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not
proven ? our corpus is badminton-only,
81        and the evidence once cited for it (RacketVision) was *refuted on verification* (it's
ball-tracking, not temporal). The
82        concept is sound on first principles (rally signal = a racquet-sport invariant) but
must be validated once a 2nd-sport
83        corpus exists. Don't cite RacketVision as evidence.
84
85      ## Language ? Python everywhere; on-device runtime is a deferred, layer-local export
shell
86      - **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer);
offline/batch, no latency hot path.
87      - **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-
process (ffmpeg, WASB-in-WSL, cloud
88        GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ?
single-digit-ms wins dwarfed by
89        ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval
```

harness + provider registry.
90   - **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate
our dev language:** train in
91   Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device
shell (Swift/Kotlin/C++/Rust)
92   runs the artifact with zero Python. **The one concrete constraint now: train with
export-clean ops** so the compression
93   target stays reachable without re-architecting. (On-device base stays on the
Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
94   - Escalation order if a real Python CPU hot path ever appears: free-threading ?
numpy/vectorize ? Cython/PyO3 ? only then a
95   separate service. **None exists today.**
96
97   ---
98
99   ## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
100
101   ### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) ·
greenlight item
102   Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index
schema (the conversational seam)
103   ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,
ending_reason/fault_tags,
104   shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105   labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL
ActivityNet-JSON · VLM clip?QA),
106   one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector`
(system-of-record; extends PR #13).
107   **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match
enforced; **no Gemini-sourced rows in any training view**.
108
109   ### M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
110   6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
111   "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club
match ? margin. **Bottleneck =
112   data + calibration, not method.** Keep growing per new sport.
113
114   ### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) ·
greenlight item
115   - **(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training
violation is
116   **`backend/eval/distill_local.py`** (it trains a classifier on Gemini-silver CSV
labels ? `output/local_rally_model.json`)
117   and the threshold distillation in `calibrate_local.py`. (NB:
`backend/eval/training.py::label_candidates` is a *pure,
118   source-agnostic* function ? there is no store by that name; the earlier
"training.label_candidates" pointer was wrong.)
119   Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the
path that regenerates them is one
120   command away ? retarget to human + open-teacher (own-model/WASB) labels; Gemini =
eval/reference/fallback only.
121   - **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint
provenance is a **commercial-ship
122   blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a
clean MIT/Apache detector swap.
123   ? **own-weights is an un-scoped NEW workstream, not ROADMAP Phase 4:** per ADR-002 our
temporal labels can't train the
124   shuttle detector (it needs per-frame coordinate labels we don't have). **Multiplies
per sport** (incl. any TT-on-WASB use).
125   Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).
126
127   ### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) · *greenlight item*
128   Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over

**ActionFormer (MIT, 1:1
129   rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not*
the Commons-Clause repo),
130   reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item):
a FLOOR of "beat the honest n=6
131   heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU**
target for "very high P/R" (e.g.
132   F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory.
Trains on the GPU/WSL box.
133
134   ---
135
136   ## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
137   Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it
becomes the rally/highlight
138   extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no
overfitting to declare victory.
139
140   ## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus +
explicit go)
141   - **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a
promising cue here** ? our E1 probe
142   (PR #35) found it weak (~2.2× discrimination, AUC ~0.6) and the foreground-audio idea
also failed this session; keep it
143   parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet
first).
144   - **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a
temporal-IoU-vs-golden reward,
145   our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on
Modal (no-access) / RunPod-Secure.
146   - **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-
hosted VLM vs Gemini) **+ a real
147   VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples
use 6?8 GPUs ? **budget is the
148   largest open risk and is currently unquantified**). Don't stand up an owned serving
stack until these clear.
149
150   ---
151
152   ## Decisions / status
153 | # | item | status |
154 |---|---|---|
155 | 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |
156 | 2 | WASB C3 ? first-class action item (own-weights / clean swap) | **owner: yes** ? |
157 | 3 | Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights) | **owner: yes** ?
|
158 | 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
159 | 5 | Rename repo off "badminton" | **owner action** ? |
160 | 6 | Conversational-QA: build seam now, punt model | **rec accepted?** |
161
162   ## Open risks / unknowns (none block Phase-0 research)
163   Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R"
target undefined · per-sport clean
164   trajectory detectors not yet identified · base-model pretraining provenance unauditable
(risk-acceptance) · real VLM
165   VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-
train (minors-excluded) for cloud
166   training · on-device 4B latency on 8 GB unmeasured.
167
168   ---
169   *Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing
discipline holds: the committed
170   next platform slice remains the async job model (single-tenant); only the M0.1?M0.4
foundation is safe to start.*

171

4. user (2026-06-14T06:50:18.869Z)

```
1       # Next steps (live) ? owned-model + multisport
2
3       **Updated:** 2026-06-13 (**rally-detection quality track**: fusion + experiment-harness
design docs merged,
4       owner-gated ? see the dedicated section below, the desktop-agent pick-up). Prior:
2026-06-10 (audit?M0?Gemini-battery session; later same day the **UI-alpha track opened**:
5       ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra` repo, local-
first delivery ? see
6       `docs/UI_ALPHA_EXECUTION_PLAN.md`; **PR-1 async jobs (#16) MERGED 2026-06-11, PR #87**;
**PR-2
7       upload/download MERGED 2026-06-11, PR #99** ? live-E2E-verified (3-part chunked upload
MD5 byte-identity +
8       compile?browser-download); next: PR-3 identity. Same day: the **8?9 quality sweep
#90?#98 landed** ? ruff/mypy/
9       coverage-floor CI lanes, SQLite conn-close fix, config unknown-key warning, hybrid-twin
collapse into
10      `trajectory_hybrid.py`, WSL tilde+abspath live-path fixes; suite 388 green; LOVO 0.626
re-verified exact).
11      **Authoritative roadmap:**
12      `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+ ADR-008 for topology). **Latest blow-by-
blow:** memory
13      `current-state.md` / `session-handoff.md`. This file = the prioritized "what to do
next," refreshed each session.
14
15      ## Where we are (one paragraph)
16      ADR-008 ratified ("repo split driven by productionization, not isolation"): training
lives in-repo (`training/`)
17      behind a CI-enforced import wall; the featurizer is single-sourced
(`backend/features/rally_seq.py`, train==serve);
18      the **Gen-0 harness ships** (`python -m training.gen0.harness --manifest
output/human_lovo_manifest.json --floor
19      eval_baselines/heuristic_lovo_n6.json --version <semver>`) and produced artifact v0.1.0
(LOVO **0.626**, floor passed,
20      sign test 5/6 wins p=0.109 ? honestly not significant, ship=False). The **n=6 owned
corpus is in the collector**
21      (262 rallies, Proprietary, commercial export = owned data only after the ShuttleSet
reclassification). The **Gemini
22      battery is done** (see table) ? the moat is quantified.
23
24      ## The quality table that now drives strategy (IoU 0.5 vs human golden labels)
25      | Path | Mahadevpura (clean-ish) | Kushagra (multi-court) |
26      |---|---|---|
27      | Heuristic (velocity windowing) | 0.657 | 0.157 |
28      | **Gen-0 owned head** (LOVO, n=6) | 0.744 | 0.561 |
29      | Two-stage WASB+Gemini refine | 0.83 | ? |
30      | **Gemini wrapper** (`gemini-flash-latest`) | **0.93** | **0.66** |
31
32      **Reading:** clean footage is commoditized (serve it with Gemini for cents). **Multi-
court drops the commodity
33      wrapper to 0.66 ? that is the owned model's ship target** (its FPs are background-game
pickups + dead-time merges,
34      the exact contrast-metric confound). Gen-0 is 0.10 behind with only n=6 ? a corpus-
growth-sized gap, not an
35      architecture-sized one. Two-stage refine = the cost-efficient serving path for LONG
tapes (uploads candidate clips
36      only); wrapper 503-flakiness (observed all session) makes the provider-fallback registry
load-bearing.
37      Baselines tracked in `eval_baselines/` (heuristic_lovo_n6.json,
gemini_wrapper_2026-06-10.json).
38
39      ## Rally-detection quality track (NEW 2026-06-13 ? design merged, owner-gated, NOT
```

started)
40   **This is the track a desktop/local agent is slated to pick up.** Three design docs
landed (PR #112 =
41   fusion + Roora v8; this session = experiment harness + handoff). All implementation is
owner-gated.
42   - **Docs:** `docs/MULTI_SIGNAL_FUSION_PLAN.md` (shuttle + person + audio cues, one
fusion layer) ·
43   `docs/EXPERIMENT_HARNESS.md` (A/B + shadow + promotion gate, zero-regression) ·
44   `docs/ROORA_LABELING_GUIDELINES.md` (v8) · `docs/HOW_RALLY_DETECTION_WORKS.md` (mental
model).
45   - **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a
rally" bug is
46   **NOT a missing capability** ? `backend/pipeline/detectors/rally_gate.py` already
implements the
47   net-crossing **"Gate A"** (`apply_gate(..., min_crossings=2)`) that kills service-
feed/held-shuttle
48   windows, but it is **only called from eval drivers** (`distill_local.py`,
`eval_partial_labels.py`,
49   `export_wasb_segments.py`, `calibrate_local.py`) and there is **no `min_crossings`
knob in
50   `IndexingConfig`** ? so production never applies it. **Wiring Gate A into the live
segmenter is the
51   single cheapest, highest-confidence quality win.**
52   - **Pick-up order (each = ONE PR off `master`, owner approval first):**
53   1. **Fusion P2 first-step ? wire Gate A into production:** add `min_crossings` to
`IndexingConfig`;
54   apply `rally_gate` in `wasb_hybrid`/`trajectory_hybrid` runtime (today it's eval-
only). Pure-logic,
55   testable in this container. *Fixes the observed bug.*
56   2. ? **DONE (2026-06-13) ? Fusion P1 ? extract the person cue:** lifted torchvision
detector + ByteTrack
57   + velocity out of `yolo_hybrid` into
`backend/pipeline/detectors/person_detector.py` (`PersonDetector`);
58   `yolo_hybrid` now orchestrates only + delegates to `self.person` (no behavior
change ? regression-safe,
59   proven by unchanged process_video tests). `tests/test_person_detector.py` added;
person_detector
60   type-clean. **The fusion segmenter (P2) reuses this producer over shuttle-seeded
windows.**
61   3. ? **DONE (2026-06-13) ? Experiment-harness P1 ? `backend/eval/experiment.py`:**
thin wrapper over
62   the EXISTING `calibration.py`, `metrics.py`, `classifier.py`, `training.py`,
`regression.gt_hash`, and
63   `database.query_candidate_segments` (no rebuilt scoring). Shipped `Variant` (+
pinned `CONTROL`),
64   `compare` (labeled A/B, LOVO-honest for learned variants), `compare_unlabeled`
(**SxS on new
65   unlabeled footage** ? agreed/A-only/B-only), `record_experiment` (leaderboard
JSON), and the DB
66   bridge; 19 tests incl. the no-regression invariant. Suite 421?440 green.
67   4. ? **DONE (2026-06-14) ? Fusion P2 ? the FusionSegmenter:**
`fusion_hybrid.py::FusionSegmenter`
68   (registry `fusion_hybrid`, **default-OFF**) seeds windows from the shuttle
substrate, persists
69   `net_crossings` (Gate A) + optional person features (`fusion_features.py`, pure-
tested) via two
70   identity hooks on the trajectory engine, with optional served Gate A/B
(`fusion.min_crossings` /
71   `min_players`, default 0) that gates the handoff while persisting the full
superset. Court mask =
72   optional `fusion.court_polygon`. Adversarially reviewed (31 agents, 19 confirmed;
NO-REGRESSION
73   verified); suite 441?467 green. Next: **P3** learned fusion (person features ?
harness `compare`
74   LOVO lift on the 6 golden videos; the eager-person-compute optimisation lands

here), then **P4**
75   audio cue via `backend/pipeline/audio/hit_detector.py`.
76   - **Discipline (from EXPERIMENT_HARNESS.md):** every new cue lands **flag-gated, default
OFF**; promote
77   to default **only on measured LOVO lift** through the `run_regression.py` gate (exit
0/1/3); pinned
78   control variant ? **rollback = config flip, never `git revert`**. The harness is ~80%
existing code.
79   - ? Dev container has no GPU/torch/cv2 ? Gate-A wiring, the person-cue extraction logic,
and
80   `experiment.py` are pure-logic/testable here; real-video confirmation is owner-side
(GPU/WSL).
81
82   ## Prioritized next steps
83   1. **[owner, in progress] GROW THE GOLDEN CORPUS** ? the single highest-leverage move;
every lever below feeds on it.
84   Priority order for new videos: **(a) multi-court badminton** (the target distribution
AND where the ship target
85   lives), **(b) ?3 matches per new sport ? table tennis first** (WASB transfers at F1
0.909; pickleball labels are
86   corpus-gold but not trainable until a clean MIT/Apache ball detector exists), (c)
capture variety per
87   `VIDEO_RECORDING_GUIDELINES.md`; **adult sessions only for the training pool**
(consent guardrail). Per video:
88   label ? `curate import-local --normalize --license Proprietary --fps <true fps>` ?
re-run the Gen-0 harness.
89   The paired sign test needs n: at n=10 with 9 wins, p?0.011 ? significance is
reachable this month.
90   2. **[owner decision] Set the ship-gate target** ? now informed: floor = heuristic 0.480
(necessary), **multi-court
91   reference = wrapper 0.66** (the number that makes the owned model worth serving),
clean-footage ceiling = 0.93
92   (not the target ? Gemini serves that tier). Suggested shape: "LOVO F1@tIoU0.5 ? 0.70
on multi-court folds with
93   paired-sign p<0.05 vs heuristic" ? owner to ratify/adjust.
94   3. **M0.4 next increment ($0/local):** swap the Gen-0 LogReg for
ActionFormer/ASFormer (MIT, minutes on the
95   2070S ? still lightweight, NOT the heavyweight Gen-2 stack) inside the existing
harness; per-video threshold
96   calibration (the LOVO-vs-oracle gap is visible per fold in the harness output).
97   4. **Multisport thread:** ingest Roora pickleball/TT CSVs (`import-local --normalize`);
merged-prototype LOVO across
98   badminton+TT once TT has ?3 matches; identify a clean pickleball ball detector. ? TT-
on-WASB serving stays gated
99   by C3 (provenance multiplies per sport).
100   5. **PMF validation (cheaper than any engineering month):** C13 academy interviews (+
the two added questions:
101   "multi-court footage nobody watches?" / "pay per-match to index it?"); camera pilot
at the warmest 2-3 academies
102   (consent at capture; demo served via the Gemini path or human-polish ? the reel
pattern exists:
103   `output/MahadevpuraSingles_highlights.mp4`). **Field kit COMPLETE (2026-06-12, owner
ratified the
104   physical-visit approach; reframed product-feedback-first per owner spec):**
105   `docs/PRODUCT_FIELD_ASSOCIATE_FLYER.md` (the recruiting flyer ? owner fills ?/contact
placeholders and recruits) +
106   `docs/FIELD_VISIT_PLAYBOOK.md` (the hired associate's visit playbook: coach + student
script, demo-after-Q6
107   rule, ground rules, Phase-2 record-and-show-back stretch goal) +
`docs/FIELD_VISIT_ANSWER_SHEET.md` (per-visit
108   capture form incl. demo-reaction + student blocks) + `docs/PMF_FIELD_SIGNALS.md`
(G/A/R anchors + **pre-registered
109   decision rules** ? PROCEED / PIVOT A-C / FALSIFIED thresholds fixed before the first
visit).
110   6. **Housekeeping:** ~~rotate the Gemini API key~~ **DONE 2026-06-10** (owner-verified

```
working; confirm old-key
111        deletion in AI Studio); **rename repo off "badminton"** (owner action,
112        in BACKLOG); collector=SoR confirmation; off-peak niceties: none pending ? the
battery completed.
113
114     ## Pointers / artifacts
115     - Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Topology: ADR-008
(`docs/archives/decisions/DECISIONS.md`) ·
116       Study: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md` · Quality history:
117       `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
118     - Harness: `training/gen0/harness.py` (one command ? train ? LOVO ? gate ? artifact
bundle; weights gitignored,
119       manifest reviewable). Featurizer: `backend/features/rally_seq.py` (FEATURE_SCHEMA v1).
120     - Corpus: collector DB (system-of-record, 6 matches/262 rallies) · eval manifest
`output/human_lovo_manifest.json`
121       (gitignored) · durable trajectories + Roora files: `C:\Users\avidu\Projects\Annotation
Setup\Collect\Trajectories\` ·
122       labels: `...\Collect\Golden Labelled\`.
123     - Gemini ops learnings: serial uploads (`indexing.ai_max_workers=1`) + chunk re-encode
124       (`indexing.ai_chunk_scale_width=1280`) + 6-attempt exponential generate retry ? a
cold/prepaid project burst-throttles
125       with a MISLEADING "credits depleted" 429; ramp gradually.
126     - ? No owned-model implementation beyond greenlit M0.x until owner greenlights the next
milestone. Committed next
127       PLATFORM slice = async job model (single-tenant).
128
```

5. user (2026-06-14T06:50:19.393Z)

```
1       # Execution policy ? per-job, self-scheduling resilience for long/flaky operations
2
3       **Status:** **DESIGN ? owner-gated, NOT started.** Motivated by a real GX010125 hang:
the Gemini
4       `generateContent` call stalled with **no client-side timeout** and blocked the whole
run. The fix is
5       not a one-off timeout ? it is a **policy** every long operation carries, so waiting /
polling / aborting /
6       resuming is declared per job, not hard-coded. Docs-only.
7
8       ## The problem (concrete)
9       `backend/providers/gemini.py::analyze_video` has **two unbounded waits**:
10      1. the file-processing poll ? `while video_file.state.name == "PROCESSING":
time.sleep(10)` (no deadline);
11      2. the `generate_content` call ? the retry loop (6 attempts, backoff) catches
**failures**, not **hangs**;
12         a request that never returns blocks forever.
13      The same shape recurs anywhere we wait on an external/long thing (WSL/WASB GPU pass,
ffmpeg, a future
14      remote compute backend). Today each site invents its own ad-hoc loop (or none). We want
one declarative
15      contract.
16
17      ## Your architecture ? validated
18      - **"Each job can specify its own [policy]."** ? Yes ? an `ExecutionPolicy` is attached
**per job**
19        (and per long sub-operation), defaulting sensibly, overridable by the caller. It is
just data on the job.
20      - **"Jobs can self-schedule."** ? Yes ? a job that must wait persists its `next_poll_at`
+ progress and
21        **re-enqueues itself**; the existing single worker re-claims it when due. No periodic
central tick.
22      - **"A master node is not necessary."** ? Correct. The repo is already master-less: a
`ThreadPoolExecutor`
23        worker (`main.py`, `max_workers=1`, single self-hosted box) drains the `jobs` table,
and a **crash-recovery
```

```
24        sweep** (`fail_stale_jobs()`, DEFERRED_HARDENING #16) reaps jobs a dead process left
behind. Self-scheduling
25        = cooperative re-enqueue; **any** worker can claim a due job, so it generalizes to N
workers
26        (PLATFORM_ARCHITECTURE pluggable compute) **without** a dedicated scheduler ? the DB
*is* the coordination.
27
28      ## The contract
29      ```python
30      class WaitMode(str, Enum):
31          WAIT_AND_RESUME = "wait_and_resume"   # DEFAULT ? bounded wait; on deadline, persist
+ reschedule
32          ABORT           = "abort"             # on deadline/hang, stop this op (fail or
partial)
33          BLOCK           = "block"             # legacy in-process bounded blocking wait (no
reschedule)
34
35      @dataclass(frozen=True)
36      class ExecutionPolicy:
37          mode: WaitMode = WaitMode.WAIT_AND_RESUME
38          poll_every_n_sec: float = 10.0        # cadence for polling an in-progress external
op
39          op_timeout_sec: float = 120.0         # HARD per-attempt deadline ? the hang-killer
(was missing)
40          max_wait_sec: float = 1800.0          # total wall budget across resumes before
giving up
41          max_retries: int = 5                  # failures (not hangs); exponential backoff +
jitter
42          backoff_base_sec: float = 3.0
43          on_timeout: Literal["reschedule", "partial", "fail"] = "reschedule"
44          resumable: bool = True                # persist partial progress so a resume
continues, not restarts
45          # JSON-serializable ? rides on the job row; a variant/job overrides any field.
46      ```
47      - **`op_timeout_sec`** is the missing piece that would have killed the GX010125 hang:
every external call
48        (upload, `files.get`, `generate_content`) runs under a hard per-attempt deadline; on
expiry the policy's
49        `on_timeout` decides ? `reschedule` (WAIT_AND_RESUME), `partial` (proceed with what's
done), or `fail`.
50      - **Hang vs failure are distinct.** Retries (`max_retries` + backoff) handle a call that
*errors*; the
51        *timeout* handles a call that *hangs*. Both are policy, not provider-baked.
52
53      ## Self-scheduling mechanism (no master)
54      A long op under `WAIT_AND_RESUME`:
55      1. runs the external call under `op_timeout_sec`;
56      2. on success ? record progress, continue;
57      3. on timeout/in-progress ? **persist progress + `next_poll_at = now +
poll_every_n_sec`, set job
58         `status=waiting`, release the worker** (don't hold a thread sleeping for minutes);
59      4. the worker loop (and the startup sweep) claims jobs whose `next_poll_at <= now` and
resumes them;
60      5. give up when cumulative wait > `max_wait_sec` ? `on_timeout` terminal action.
61
62      This needs **only** two `jobs`-table columns (`next_poll_at`, `progress` JSON ?
`progress` already exists)
63      and a "claim due jobs" query. No timer service, no leader, no master. On one box it is
one worker draining a
64      priority-by-due-time queue; on many boxes it is the same query with a row-claim (`UPDATE
? WHERE status=
65      'waiting' AND next_poll_at<=now`), so a job migrates to whoever is free.
66
67      ## Resumability (why WAIT_AND_RESUME is the default)
68      The op persists **what it already finished** so a resume does *not* redo it. For the AI
```

```
handoff that means
69      **per-window confirmation state**: confirmed windows are written as they complete; a
resume processes only
70      the unconfirmed ones. The GX010125 run is the worked example ? instead of the 7-min cap
abandoning ~10
71      windows and losing them, WAIT_AND_RESUME would persist the confirmed windows and
reschedule the rest, so the
72      reel becomes exhaustive across resumes with **no wasted Gemini spend**. `ABORT` (or
`on_timeout=partial`) is
73      the opt-out for "good enough now" (what the one-off script does today).
74
75      ## Phasing (owner-gated; ONE PR per slice, off `master`, default = current behaviour)
76      - **P0 ? this doc.**
77      - **P1 ? `ExecutionPolicy` + bound the two Gemini waits (the actual hang fix).** Add the
dataclass; thread a
78        policy into `GeminiProvider.analyze_video`; run upload / `files.get` poll /
`generate_content` under
79        `op_timeout_sec` (a thread-future wrapper or the genai client's request timeout) with
the poll deadline.
80        Default policy reproduces today's behaviour **except** hangs now terminate per
`on_timeout` instead of
81        blocking forever. Pure-logic, unit-testable (a fake provider that hangs ? policy
aborts/reschedules).
82      - **P2 ? per-job policy + self-scheduling** on the `jobs` table (`next_poll_at`, claim-
due query,
83        `status=waiting`); the executor honours WAIT_AND_RESUME by releasing + rescheduling
instead of blocking.
84      - **P3 ? resumable AI handoff** (persist per-window confirmation; resume processes only
unconfirmed) ? makes
85        the GX010125-style run exhaustive across resumes.
86      - **P4 ? apply the same policy to the WSL/WASB GPU pass + ffmpeg** (the other long ops).
87
88      ## Trade-offs / notes
89      - **Default WAIT_AND_RESUME, but bounded** ? `max_wait_sec` guarantees forward progress
/ eventual give-up;
90        it never waits truly forever (the current bug).
91      - **Idempotent resumes** ? a resumed op must not double-spend (e.g. re-upload a clip
already confirmed);
92        P3's per-window state is what makes that safe.
93      - **Cost control** ? `op_timeout_sec` + `partial` is the cheap path; `WAIT_AND_RESUME`
trades wall-clock for
94        completeness without re-spending. Both are one field on the job.
95      - **No new infra** ? rides the existing `jobs` table + single-worker executor + crash
sweep; the DB is the
96        coordinator, so it scales to multi-worker (PLATFORM_ARCHITECTURE) with no master
added.
97
98      **Links:** the async-job substrate is `docs/DEFERRED_HARDENING_PLAN.md` §3 (#16); multi-
worker future is
99      `docs/PLATFORM_ARCHITECTURE.md`; the motivating hang is the GX010125 run
100     (`scratch/run_gx010125_full.py`, where a 7-min cap had to be hand-rolled).
101
```

6. user (2026-06-14T06:50:20.686Z)

```
1       # Roora Annotation Guidelines ? Rally Golden Set
2
3       **Version: v8 (2026-06-13).** Supersedes the older Drive copies (v3/v5/v6/v7) **and**
the internal
4       "DRAFT v2" label ? those numbers never matched each other; **v8 is the single label to
use with
5       Roora going forward.** Content is otherwise the finalized 2026-06-12 guideline plus the
6       fusion-driven additions marked **`? ADDED (fusion)`** below.
7
8       > **?? Canonical home / provenance.** The source of truth for this guideline is the
```

sibling
9   > **`sports-data-collector`** repo: `docs/annotation/` (`ROORA_BRIEF.md` +
`ANNOTATION_GUIDE.md` +
10   > intake-manifest & NDA templates), generated from `sports-data-collector@ca61405`, and
ingested by
11   > `python -m src.cli.curate import-local`. **This file is a mirror kept here** because
the
12   > multi-signal-fusion features that motivate the new fields (court calibration,
singles/doubles) live
13   > in *this* repo ? see MULTI_SIGNAL_FUSION_PLAN.md and
14   > GOLDEN_SET_VENDORING.md (ADR-006). **Any change here must
be synced back
15   > to the collector's `docs/annotation/` before it goes to Roora.** The `? ADDED
(fusion)` callouts are
16   > exactly the deltas to port.
17   >
18   > **`TODO(avidullu)` markers are preserved** ? they are owner-only items (Drive links,
intake
19   > manifest, NDA counsel review) and must be filled before sending to Roora. Search for
`TODO(avidullu)`.
20
21   Brief (vendor spec) + Illustrated Guide (rater how-to), in one doc.
22
23   ---
24
25   # PART A ? Roora Annotation Brief (vendor-facing spec)
26
27   **Project:** racket-sport rally segmentation golden / regression dataset.
28   **Sports in scope:** Badminton, Pickleball, Table Tennis (each in singles AND doubles).
29
30   This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV that
imports
31   directly via `python -m src.cli.curate import-local`.
32
33   ## 0. What we need in one sentence
34
35   For each provided match video, produce ONE CSV file listing EVERY rally with its start
time, end
36   time, number of shots, and how it ended ? EXCLUDING all dead time.
37
38   ## 1. Deliverable format (canonical)
39
40   One CSV per video (UTF-8), named exactly `<video_id>.csv` (we supply the `video_id`).
One row per
41   rally, chronological, header row required.
42
43   **Columns:**
44
45 | Column | Type / values | Meaning |
46 |---|---|---|
47 | `rally_number` | integer, 1-based, chronological ? no gaps, no repeats | rally index |
48 | `start_mmss` | `m:ss.mmm` (e.g. `1:12.40`) | rater-entered serve-contact time |
49 | `end_mmss` | `m:ss.mmm` | rater-entered ball/shuttle-dead time |
50 | `start_time` | decimal **seconds** (e.g. `72.40`) ? **AUTO-FILLED** from `start_mmss`
| machine time |
51 | `end_time` | decimal **seconds** ? AUTO-FILLED from `end_mmss` | machine time |
52 | `shots_count` | integer | total racket/paddle/bat contacts (serve = shot 1) |
53 | `ending_reason` | `winner` \| `forced_error` \| `unforced_error` \| `service_fault` \|
`let` \| `other` | WHY it ended |
54 | `last_shot_outcome` | `in` \| `net` \| `out` \| `n_a` | WHERE the last shot landed
(`n_a` for service_fault/let) |
55 | `score_before` | optional, `"A-B"` ? blank if not confidently readable | score before
|
56 | `score_after` | optional, `"A-B"` | score after |
57 | `notes` | optional free text ? **REQUIRED when `ending_reason = other`** | edge-case

```
note |
58
59      **Times (mm:ss ? decimal seconds):** raters type only `start_mmss` / `end_mmss`. The
template Google
60      Sheet fills `start_time` / `end_time` with this formula (shown for `start_time`, row 2):
61
62      ```
63      =IF(B2="","", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 +
VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
64      ```
65
66      On "Download as CSV" the computed decimal seconds export. (If your tool already shows
decimal
67      seconds, type those into `start_time` / `end_time` and leave the mm:ss columns blank.)
68
69      **Example (badminton):**
70
71      ```
72      rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot
_outcome,score_before,score_after,notes
73      1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
74      2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
75      3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no
pressure)
76      ```
77
78      **Critical format rules:**
79      - `start_time` / `end_time` that reach us MUST be decimal seconds (the formula
guarantees this).
80      - Times are relative to the **START OF THE FILE WE PROVIDE**. Do NOT trim, clip, or re-
encode the
81        video ? our pipeline identifies a video by its file checksum. (The file we share may
be an
82        **annotation-optimized copy** of the original ? lighter and faster to frame-step, but
83        frame-for-frame identical. Treat it exactly like the original: annotate it as-is.)
84      - `end_time > start_time` for every row.
85      - No overlaps: `end_time[N] <= start_time[N+1]`.
86      - Everything between one rally's `end_time` and the next rally's `start_time` is DEAD
TIME and gets
87        NO row.
88
89      ## 2. The three non-negotiable rules (full detail + pictures in the Guide)
90
91      1. **IGNORE ALL DEAD TIME.** Only label active play.
92      2. **INCLUDE EVERY RALLY ? EXCLUDE NONE.** Every served point, including service faults,
1?2 shot
93         rallies, and lets/replays. A missed rally is the worst error in this project.
94         - **EXCEPTION:** warm-up / knock-up is NOT a rally ? pre-match and between-games
warm-up looks like
95           rallying but isn't scored play, so don't label it. Tell it apart by: no real serve;
cooperative
96           (not competitive) hitting; the score not progressing; often several shuttles/balls
in use (full
97           how-to in the Guide, Part 2). When unsure if the match has started, flag it.
98         - **SECOND EXCEPTION:** a rally already in progress when the file starts (or still in
progress when
99           it ends) gets NO row ? its true start (or end) is outside the footage and must
never be guessed
100          or extrapolated. Note it in the delivery instead.
101     3. **A SHOT = ONE CONTACT** with the racket/paddle/bat. Serve = shot 1. Bounces and net-
clips are not shots.
102
103     ## 3. Scope & phasing
104
105     - **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table
```

tennis), each
106   < 3 min with ~10?12 rallies. This is the paid calibration round; we review jointly and
lock the
107   Guide before scale-up.
108   - **Phase B ? Scale (after a successful pilot):** ~12?15 videos PER SPORT (~36?45
total), each ~30 min.
109   Same format, rules, and template.
110
111   Doubles are included across the program. Rally rules are identical to singles; the
badminton- and
112   pickleball-specific doubles notes are in the Guide.
113
114   ## 4. Quality bar (acceptance criteria)
115
116   Automated validation (no overlaps, positive durations, schema-valid) runs on every file,
then a ~20%
117   human spot-check. A file is returned for rework (in scope) if it misses:
118
119   - **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
120   - **Boundaries:** `start_time` / `end_time` within **±0.3 s** of true serve-contact /
landing.
121   - ?? Boundaries that all sit on a frame-sampling grid (e.g. every 30th frame)
STRUCTURALLY FAIL this
122   bar and are detected automatically ? the whole file is returned for rework. See §7
for the
123   required method.
124   - **`shots_count`:** exact for rallies ? 15 shots; ±1 allowed for longer/faster rallies.
125   - **`ending_reason`:** one of the six allowed values, per the Guide's decision tree.
126   - **Format:** decimal seconds present, valid columns, non-overlapping, chronological.
127
128   > **? ADDED (fusion) ? court calibration as a day-1 quality lever.** The owner's
pipeline produced a
129   > bad highlight where a held shuttle was mislabeled as a rally. The boundary definition
(§ Part 3:
130   > start = serve **contact**, not the toss/hold) already excludes that ? so faithful
boundaries are
131   > themselves a quality checkpoint. The one NEW field we need (next section) is **court
calibration**,
132   > which lets the detector reason about court occupancy and net-crossings.
133
134   ## A. Court calibration & intake manifest   ? ADDED (fusion)
135
136   > **? ADDED (fusion).** *Why:* the multi-signal detector
137   > (MULTI_SIGNAL_FUSION_PLAN.md) needs to know **where the
court is** and
138   > **where the net is** to (a) ignore people outside the court, (b) require ?2 players on
court for a
139   > rally, and (c) count shuttle net-crossings. This is a **one-time, per-video**
annotation ? cheap,
140   > high-leverage, and **required in deliverables from day 1.**
141
142   **Per-video, recorded once in the intake manifest (NOT in the per-rally CSV):**
143
144 | Manifest column | Type | Meaning |
145 |---|---|---|
146 | `video_id` | string | (existing) we supply it |
147 | `fps` | number | (existing) we supply it |
148 | `court_corners` | 4 points `(x,y)` in pixels, clockwise from the near-left corner |
the playable court rectangle |
149 | `net_line` | 2 endpoints `(x,y)` in pixels | the net, left post ? right post |
150 | `singles_or_doubles` | `singles` \| `doubles` | sets the expected on-court player
count (2 vs 4) |
151
152   - Pick **one clear frame** with the empty/visible court and click the 4 corners + 2 net
endpoints.

- If the camera does not move for the whole video (the normal case), one calibration covers the file.
  If the camera is moved mid-file, flag it ? we'll split the file.
- Worked example (badminton, 1920×1080): `court_corners=[(360,980),(1560,980),(1330,300),(590,300)]`,
  `net_line=[(470,640),(1450,640)]`, `singles_or_doubles=singles`.

(After the first batch we may add more per-shot fields ? see the §B appendix ? and renegotiate price.)

5. Per-sport quick reference (full version + examples in the Guide)

- **Badminton (singles & doubles)** ? start: racket?shuttle serve contact · end: shuttle floors /
  fault · shot: each racket?shuttle contact.
- **Pickleball (singles & doubles)** ? start: paddle?ball serve contact · end: double bounce / out /
  net / fault · shot: each paddle?ball contact (floor bounces are NOT shots; two-bounce rule is normal play).
- **Table tennis** ? start: racket?ball serve strike (not the toss) · end: failed legal return · shot:
  each racket?ball contact (table bounces are NOT shots). Net-cord serve = let.

6. `ending_reason` values (decision tree in the Guide)

This vocabulary attributes WHY the point ended; it is shared across all net-separated sports
(badminton, pickleball, table tennis, tennis). Record WHERE the last shot physically landed in the
separate `last_shot_outcome` column (`in` | `net` | `out` | `n_a`) ? e.g. a forced error that flew
long is `ending_reason=forced_error, last_shot_outcome=out`.

- `winner` ? last shot landed IN and the opponent made no legal contact (a clean winner).
- `forced_error` ? the losing player erred (netted / out / missed) UNDER PRESSURE ? stretched,
  wrong-footed, or rushed by the opponent's strong shot.
- `unforced_error` ? the losing player erred on a ROUTINE, UNPRESSURED ball they had time and position
  to make.
- `service_fault` ? the serve itself failed / was illegal.
- `let` ? the point was replayed.
- `other` ? anything else / cannot tell. MUST add a note.

7. Capture method (required for boundary frames)

Use any tool that shows current time (`mm:ss.mmm` or seconds, or frame + known fps) and can step one
frame forward/back: a frame-steppable player (VLC frame-step) or a timeline tool (Label Studio /
CVAT). The deliverable is the CSV in the schema above ? or, if you work in CVAT, the CVAT export as-is
(we recompute each rally's time from its box frame range; seconds = `frame_number / fps`, fps provided
per video).

Boundary placement is the non-negotiable part:
- Set up the CVAT task at **FRAME STEP 1** on the file we provide (it is encoded to make this fast ?
  frame-stepping and back-scrubbing stay snappy).
- The FIRST box of every rally goes on the EXACT serve-contact frame; the LAST box on the EXACT
  shuttle/ball-dead frame. Frame-step to find them.
- Boxes IN BETWEEN may stay sparse (e.g. every 30th frame) ? only the two boundary

frames must be exact.
    - Self-check before submitting: every start/end within ±0.3 s of the true frame (Guide).

    ## 8. Logistics & contacts

    - **TODO(avidullu):** paste the Google Drive folder link where source videos are shared with Roora.
    - Per-video fps + `video_id` (+ the calibration columns from §A): provided in the intake manifest
      (template `INTAKE_MANIFEST_TEMPLATE.csv`). **TODO(avidullu):** fill the manifest (one row per video)
      and share its link.
    - **TODO(avidullu):** paste the Drive folder link where Roora uploads completed `<video_id>.csv` files.
    - Blank annotation template Sheet (mm:ss ? seconds formula from §1 pre-filled):
      TODO(avidullu): create it and paste the link.
    - **TODO(avidullu):** add the escalation channel (email / WhatsApp group) for edge-case questions.
    - NDA before any footage is shared ? template `NDA_TEMPLATE.md`. **TODO(avidullu):** have counsel
      review, fill the blanks, and have Roora sign.
    - Roora contact on file: info@rooragrp.com (HSR Layout, Bangalore).

    > **? ADDED (fusion) ? data-handling guardrails (put in the brief / contract):**
    > - **Exclude minors entirely.** If footage contains children, that video does **not** enter the
    >   labeled pool ? do not annotate it; flag it back to us.
    > - **No biometric / identity labeling.** Refer to players only by **per-video pseudonyms** (e.g.
    >   "near side" / "far side") ? never a real name, and never a cross-video person ID. We do **not**
    >   collect gait or biometric annotations.
    > - **No pose / skeleton / keypoint annotation** is requested.
    > - **Work-for-hire IP assignment** (atop the NDA): the labels Roora produces are assigned to us as
    >   owned work product.

    ## 9. Pricing request (please quote)

    1. Price per rally and per minute of footage.
    2. Pilot price (3 short videos, ~30?36 rallies) as a paid calibration round.
    3. Indicative scale price for ~36?45 longer videos.
    4. Turnaround for the pilot and throughput (rallies/day) at scale.
    5. QA process and rework policy.
    6. Whether annotators have racket-sport familiarity (helpful, not required).
    7. Confirmation you can deliver the CSV schema above **(incl. the §A intake-manifest calibration columns).**

    # PART B ? Illustrated Annotation Guide (rater how-to)

    Sports: Badminton, Pickleball, Table Tennis (singles AND doubles).

    > **Illustrations:** boxes marked `[SCREENSHOT: ...]` are placeholders.
    > **TODO(avidullu):** replace every `[SCREENSHOT: ...]` placeholder with a real frame grab from the
    > pilot videos before sending this guide to Roora.

    ## Part 1 ? What you are producing

    One CSV per video (from the template Sheet), one row per rally:
    `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes`.

You type: `rally_number`, `start_mmss`, `end_mmss`, `shots_count`, `ending_reason`, `last_shot_outcome`
(+ optional score/notes). The `start_time` / `end_time` decimal-second columns FILL THEMSELVES via a
formula already in the template.

For every rally you record:
- When it started (serve contact) ? `start_mmss` (`m:ss.mmm`, e.g. `1:12.40`)
- When it ended (shuttle/ball dead) ? `end_mmss`
- How many shots happened ? `shots_count`
- Why it ended ? `ending_reason` · Where the last shot landed ? `last_shot_outcome`

Everything else (gaps between rallies) you IGNORE.

Part 2 ? The three golden rules

RULE 1: IGNORE ALL DEAD TIME. A rally is CONTINUOUS active play. The moment play stops, the rally
is over. The time until the next serve is DEAD TIME and is never labeled.

```
  DEAD        RALLY 1                 DEAD             RALLY 2            DEAD
 (ignore) |==================|     (ignore)     |==============|     (ignore)
          ^                  ^                 ^              ^
       serve              shuttle            serve         shuttle
       contact            lands             contact        lands
       =start_1           =end_1            =start_2        =end_2
```

Dead time includes: walking back to serve, picking up the shuttle/ball, wiping sweat, bouncing the
ball before serving, the score being announced, replays/slow-mo, crowd or coach shots, changeovers,
towel/medical timeouts, and PRE-MATCH / BETWEEN-GAMES WARM-UP (knock-up) ? see the Rule 2 caveat below.

RULE 2: INCLUDE EVERY RALLY ? EVEN THE UGLY ONES. Every served point gets a row, including:
- Service faults (serve into net/out/illegal) ? usually `shots_count = 1`, `ending_reason = service_fault`.
- Very short rallies (serve + one error).
- Lets / replays ? `ending_reason = let`; the replayed point is its OWN next row.

A missed rally is the worst error in this project ? the model must learn that short and failed rallies
are still rallies.

> ?? **BUT WARM-UP / KNOCK-UP IS NOT A RALLY** ? a common trap, because Rule 2 says "include
> everything." Pre-match and between-games warm-up LOOKS like rallying but is not scored play; do not
> label it. How to tell it apart:
> - **No real serve.** Real play starts with a proper serve from the service position (diagonal,
>   behind the service line). Warm-up has players feeding cooperatively from mid-court with no serve.
> - **Cooperative, not competitive.** Gentle hits straight to each other; nobody is trying to win the
>   point (no smashes-to-win, no scrambling/diving).
> - **Score isn't progressing.** No umpire / scoreboard not running / players casually retrieve and
>   re-feed; the score doesn't change. Often several shuttles/balls in use at once.
> - **Per sport:** badminton & pickleball ? real play starts with the underhand serve from the service
>   court / behind the baseline; table tennis ? real play starts with the tossed, two-

bounce serve.
299   >
300   > When unsure: find the first proper serve where the score starts to progress ? labeling
begins there.
301   > If you genuinely can't tell whether the match has started, flag the clip to us ? don't
guess.
302
303   **RULE 3: A SHOT = ONE CONTACT WITH THE RACKET / PADDLE / BAT.** Count every time the
shuttle/ball
304   touches a racket, paddle, or bat. The serve is shot 1.
305   - Count: any racket/paddle/bat contact (serve, return, smash, block?).
306   - Don't count: floor bounces, table bounces, net clips, the toss before a serve.
307
308   5-shot example: serve(1) ? return(2) ? hit(3) ? hit(4) ? winner(5) ? `shots_count = 5`,
309   `ending_reason = winner`.
310
311   ## Part 3 ? Where exactly does a rally start and end?
312
313   **START (`start_mmss`):** the exact frame the server's racket/paddle/bat CONTACTS the
shuttle/ball.
314   **Not the toss, not the wind-up ? the contact frame.** `[SCREENSHOT: serve-contact
frame]`
315
316   > **? NOTE (fusion):** this is the field that fixes the "held-shuttle" false positive ?
a player
317   > holding/flicking the shuttle before serving is **dead time**, so it never gets a row.
Faithful
318   > serve-**contact** boundaries are exactly what the detector is graded against.
319
320   **END (`end_mmss`):** the exact frame the shuttle/ball becomes DEAD:
321   - Badminton: shuttle touches the floor, or play stops on a fault.
322   - Pickleball: the second floor bounce, or the frame the ball hits the net / lands out.
323   - Table tennis: the second bounce on one side, a non-serve net hit, or off the table.
324
325   `[SCREENSHOT: end frame]`
326
327   If occluded/off-screen, pick the closest clear frame and add a note. **Boundary
tolerance: ±0.3 s
328   (~±9 frames at 30 fps).**
329
330   ## Part 4 ? `ending_reason`: how to choose
331
332   Pick exactly ONE value for how the rally ended. Stop at the first match:
333   1. Did the SERVE itself fail / was illegal (and ended the point)? ? `service_fault`
334   2. Was the point REPLAYED (let / interruption)? ? `let`
335   3. Did the final shot land IN and the opponent make NO legal contact? ? `winner` (a
clean winner)
336   4. Did the losing player make an error (netted / out / missed) UNDER PRESSURE ?
stretched,
337   wrong-footed, or rushed by a strong shot? ? `forced_error`
338   5. Did the losing player make an error on a ROUTINE, UNPRESSURED ball they had the time
and position
339   to make? ? `unforced_error`
340   6. None of the above / cannot tell ? `other` (add a note)
341
342   **FORCED vs UNFORCED ? the one judgment call.** Decide using the opponent's PRECEDING
shot:
343   - `forced_error` = the opponent's shot CAUSED the miss (pace, placement, deception; the
player barely
344   reached it or was pulled out of position).
345   - `unforced_error` = a clean, makeable ball the player missed on their own.
346   - When genuinely unsure between the two, choose `unforced_error` and add a brief note.
347
348   Notes:
349   - A clean winner means NO legal contact by the opponent. If they got a racket/paddle/bat

on it but
350   netted or hit out, that is a `forced_error` / `unforced_error` (their shot), not a
winner.
351   - A serve into the net is `service_fault` (rule 1 beats everything).
352
353   **THEN SET `last_shot_outcome`** ? where the last shot physically went. This is a
SEPARATE,
354   observational column (no judgment): just look at the last shot.
355   - `in` ? landed in (winners; also a put-away the opponent couldn't reach).
356   - `net` ? went into the net.
357   - `out` ? landed outside the court / past the lines.
358   - `n_a` ? not applicable: use for `service_fault` and `let`.
359
360   So each rally gets both: WHY (`ending_reason`) and WHERE (`last_shot_outcome`) ? e.g. a
smash winner =
361   `winner, in`; a clear forced long = `forced_error, out`; a routine ball netted =
`unforced_error, net`.
362
363   ## Part 5 ? Per-sport detail
364
365   ### Badminton
366   - **Serve:** underhand, below the waist, diagonal. Start = racket?shuttle contact.
367   - **Rally end:** shuttle hits the floor, or play stops on a fault.
368   - **Shots:** every racket?shuttle contact; serve = shot 1.
369   - **Service faults** (1-shot rally, `service_fault`): serve into net, serve outside
service court,
370   contact above waist, foot fault.
371
372   Worked example: `[SCREENSHOT: badminton serve]`
373   - Rally 4: serve `1:28.30`, long rally, smash winner, shuttle lands `1:44.10`, 12
contacts.
374   `4, 1:28.30, 1:44.10, (auto), (auto), 12, winner, in, 3-5, 4-5,`
375   - Rally 5: serve `1:57.00` straight into the net.
376   `5, 1:57.00, 1:57.90, (auto), (auto), 1, service_fault, n_a, 4-5, 4-6,`
377
378   **Badminton ? doubles (2 per side).** All rally rules are IDENTICAL to singles:
379   - start = serve contact; end = shuttle down / fault; a SHOT = any racket?shuttle contact
by EITHER
380   player on a side.
381   - `shots_count` = TOTAL contacts across both teams in the rally. Do not track which
player hit ? just
382   count every contact.
383   - Net exchanges (drives, flat pushes) come fast ? frame-step so you neither miss nor
double-count.
384   - Only the designated receiver returns the serve; if the wrong player returns, the rally
ends on a
385   fault ? still a row (`service_fault` if it's a serve/receiving-position fault, else
`other` + note).
386   - Two teammates may NOT both hit the shuttle in succession (double-hit). Two contacts on
the SAME side
387   back-to-back = a fault ending the rally ? count both contacts, then the rally ends.
388
389   ### Pickleball
390   - **Serve:** underhand, paddle below waist, hit diagonally; must clear the non-volley
zone (the
391   "kitchen"). Start = paddle?ball contact.
392   - **Two-bounce rule:** the serve must bounce once in the receiver's court and the return
must bounce
393   too, before either side may volley. These bounces are NORMAL play, not rally ends, and
bounces are
394   NOT shots.
395   - **Rally end:** ball bounces twice, lands out, hits the net (fails to cross), or a
fault (e.g.
396   volleying in the kitchen).
397   - **Shots:** every paddle?ball contact only; serve = shot 1; floor bounces are not

shots.
398   - **Service faults** (`service_fault`): serve into net, out/long, short in the kitchen,
foot fault.
399
400   Worked example: `[SCREENSHOT: pickleball serve + kitchen line]`
401   - Rally 2: serve `0:33.10`; serve bounces, return bounces, a few volleys; receiver nets
an easy one at
402   `0:41.85`; 6 paddle contacts.
403   `2, 0:33.10, 0:41.85, (auto), (auto), 6, unforced_error, net, 1-0, 2-0, easy ball
netted`
404
405   **Pickleball ? doubles (the standard format, 2 per side).** All rally rules IDENTICAL to
singles:
406   - start = serve contact; a SHOT = any paddle?ball contact by EITHER teammate; floor
bounces still not
407   shots; two-bounce rule still applies.
408   - `shots_count` = TOTAL paddle contacts across both teams; don't track which player.
409   - Serving sequence (FYI only ? you do NOT annotate it): both partners serve before the
serve passes to
410   the opponents, except the very first service turn of the game. The called score has
three numbers.
411   Ignore for annotation; just mark boundaries / shots / ending.
412   - If both teammates contact the ball in succession, that's a fault ? rally ends; count
the contacts,
413   set the ending per the fault.
414
415   ### Table tennis
416   - **Serve:** ball tossed up from an open palm, then struck so it bounces once on each
side. Start = the
417   racket?ball strike (not the toss).
418   - **Rally end:** a player fails a legal return ? ball double-bounces on their side, goes
off the table,
419   or is hit into the net on a non-serve shot.
420   - **Shots:** every racket?ball contact only; serve = shot 1; table bounces are not
shots.
421   - **Let:** a serve that clips the net but is otherwise legal ? the point is replayed.
422   `ending_reason = let`, and the replay is its own next row.
423   - **Service faults** (`service_fault`): missed/illegal toss, serve into net, serve off
the table.
424   - **Speed warning:** contacts can be < 0.2 s apart ? step frame-by-frame to count shots.
425   - **Doubles** (if a clip appears): partners must hit in STRICT ALTERNATION and serve
diagonally; a
426   missed alternation is a fault that ends the rally. Same rule ? count every racket?ball
contact as a
427   shot; `shots_count` is the team total.
428
429   Worked example: `[SCREENSHOT: table-tennis serve contact]`
430   - Rally 9: serve `4:10.40`; fast 7-contact rally; forehand winner, opponent no contact;
dead `4:14.05`.
431   `9, 4:10.40, 4:14.05, (auto), (auto), 7, winner, in, 6-9, 6-10,`
432   - Rally 10: serve `4:22.10` clips the net (otherwise good) ? let.
433   `10, 4:22.10, 4:23.00, (auto), (auto), 1, let, n_a, 6-10, 6-10, net-cord serve,
replayed`
434
435   ## Part 6 ? Edge cases & FAQ
436
437   - **Two rallies with almost no gap?** Still two rows. `end_time[N] <= start_time[N+1]`,
never overlap.
438   - **Interrupted & replayed?** Label the interrupted attempt as its own row
(`ending_reason=let`), then
439   the replayed point as the next row.
440   - **Serve tossed but caught / re-served (no contact)?** No contact = no rally yet. Start
at the actual
441   serve contact; don't label the aborted toss.
442   - **Opponent reached the ball but netted/hit out?** That's a

`forced_error`/`unforced_error` (their
443   shot), not winner ? forced if your shot pressured them, unforced if it was routine.
444   - **Can't read the scoreboard?** Leave `score_before`/`score_after` blank. Never guess.
445   - **"Let" where the rules say play on?** If play continued, it's NOT a let ? keep one
rally and judge
446   the real ending.
447   - **Warm-up / knock-up / practice points?** Not real rallies ? do not label them. Start
at the first
448   real, scored serve. See the Rule 2 caveat (Part 2). When unsure whether the match has
started, flag
449   the clip ? don't guess.
450   - **Doubles?** Same rules ? a shot is any one racket/paddle contact by whichever player
hits it. Never
451   attribute `shots_count` to an individual, just total the contacts.
452   - **Server completely whiffs (no contact)?** Rare. True zero contact ? `shots_count =
0`,
453   `ending_reason = service_fault`, add a note. A graze = 1 contact = 1 shot.
454
455   ## Part 7 ? Self-check before you submit (per video)
456
457   - [ ] `rally_number` is 1,2,3? no gaps or repeats.
458   - [ ] Every `start_time < end_time` (check the auto-filled decimal columns populated).
459   - [ ] No overlaps: `end_time[N] <= start_time[N+1]`.
460   - [ ] Times entered as `m:ss.mmm`; decimal `start_time`/`end_time` auto-filled and look
right.
461   - [ ] No dead-time rows (scan the gaps ? dead time, not missing rallies).
462   - [ ] Every served point present, including service faults and lets.
463   - [ ] `shots_count` looks right (re-step any rally you rushed; doubles = team total).
464   - [ ] `ending_reason` in {winner, forced_error, unforced_error, service_fault, let,
other}; notes
465   filled when `other`.
466   - [ ] `last_shot_outcome` in {in, net, out, n_a} (`n_a` for service_fault/let).
467   - [ ] File downloaded as `<video_id>.csv`, header row present, UTF-8.
468   - [ ] Did NOT trim / re-encode / rename the source video.
469   - [ ] **? (fusion)** intake-manifest row filled: `court_corners`, `net_line`,
`singles_or_doubles`.
470
471   ## Part 8 ? Mini glossary
472
473   - **Rally** ? one continuous passage of play from serve contact to the shuttle/ball
dead.
474   - **Shot** ? one contact between the shuttle/ball and a racket/paddle/bat (either
teammate in doubles).
475   - **Dead time** ? any time the shuttle/ball is not in active play. Never labeled.
476   - **Service fault** ? a failed/illegal serve that ends the point. Still a labeled rally.
477   - **Let** ? a replayed point. Labeled as its own rally with `ending_reason=let`.
478   - **Winner / forced_error / unforced_error** ? the final shot was a clean winner
(opponent made no
479   legal contact) / the loser's error was forced by a pressuring shot / the loser's error
was on a
480   routine, unpressured ball.
481   - **start_time / end_time** ? decimal seconds from the start of the provided file (auto-
filled from mm:ss).
482   - **? court_corners / net_line / singles_or_doubles** ? per-video calibration in the
intake manifest (§A).
483
484   ---
485
486   ## Appendix B ? Tier-B extensions (NOT in the day-1 deliverable)   ? ADDED (fusion)
487
488   > Pilot these on the 3-video calibration round, then renegotiate price before scaling.
**Do not start
489   > these without explicit instruction** ? they are listed so Roora can quote them.
490   >
491   > - **Per-shot hit timestamps** ? a timestamp for *each* racket/paddle/bat contact (not

```
just the
492     >   `shots_count` total). Extends the §7 CVAT method (box every contact, not only the
two boundary
493     >   frames). Trains/validates the audio hit-detector and shot-cadence features.
494     > - **Per-shot type** ? for each contact, a label from {serve, clear, drop, smash, net,
drive, lift}.
495     >   Needs its own short guideline + an inter-annotator agreement check.
496
```

7. assistant (2026-06-14T06:50:22.670Z)

I have all four docs. Let me compile the structured findings.

8. user (2026-06-14T06:51:31.720Z)

Structured output provided successfully

9. assistant (2026-06-14T06:51:36.361Z)

I've extracted and returned the planned work and corpus/labeling story via StructuredOutput. The four docs were all read in full (Roora skimmed for the oracle/calibration deltas), and the findings cover the owned-model roadmap, the execution-policy resilience framework, the concrete next experiments, and the Roora-as-oracle / owner-grown-proprietary-minors-excluded corpus framing as requested.